

AI Browser Extension for Phishing Detection

Final Project Report

23K-0714 S M Attique Ur Rehman | 23K-0574 Anfas Ali | 23K-0511 Asher Ahmed

Executive summary. This project implements a Chrome/Edge browser extension that detects phishing URLs in real time using a Flask backend and a machine learning classifier. It warns users before they enter sensitive information and includes user feedback to reduce false positives.

1. Introduction

Phishing websites imitate trusted services to steal credentials, card details, and personal information. The goal of this project is to demonstrate how artificial intelligence can support safer browsing by classifying URLs as legitimate or suspicious while the user browses the web.

The solution combines machine learning, URL feature engineering, a Flask prediction API, and a Manifest V3 browser extension. The extension performs real-time checks, displays a popup status, and shows a full-page warning when a phishing risk is detected.

2. Objectives

- Detect phishing websites in real time from the browser.
- Train and evaluate machine learning models for URL classification.
- Warn users clearly before they enter sensitive information.
- Reduce false positives through trusted domains and user feedback.
- Demonstrate a practical AI application in cybersecurity.

3. System Architecture

The system is divided into five main parts. The extension captures the current tab URL, the backend predicts risk, the model performs classification, and feedback files allow user corrections to improve future scans.

Component	Main Files	Purpose
Browser Extension	extension/manifest.json, background.js, popup.js, content.js	Captures visited URLs, displays scan status, and shows phishing warnings.

Component	Main Files	Purpose
Flask Backend	backend/app.py	Provides /predict and /mark-legitimate APIs for scanning and user feedback.
Feature Extraction	backend/features.py	Converts URLs into numeric ML features such as entropy, suspicious keywords, IP address flags, and query structure.
Model Training	backend/train_model.py	Trains and compares ML models, then saves the best classifier as phishing_model.pkl.
Datasets	dataset/*.csv, real_phishdataset_balanced.xlsx	Stores real phishing data, trusted legitimate domains, and user feedback.

4. Dataset and Feature Engineering

The project uses the public ESDAUNG/PhishDataset balanced dataset, based on PhishTank phishing URLs and legitimate URL sources. It is normalized into url,label format. The project also includes trusted legitimate domains and user feedback data to reduce false positives for well-known services.

Key URL Features

- Length-based features: URL length, domain length, path length, query length, and longest token length.
- Character-based features: dot count, hyphen count, digit count, special character ratio, and entropy.
- Security indicators: HTTPS usage, IP address in hostname, suspicious TLD, URL shortener, and encoded characters.
- Phishing patterns: suspicious keywords, brand keywords in subdomains, executable file extensions, and redirect-like paths.

5. Machine Learning Model

The training script evaluates Logistic Regression, Decision Tree, Random Forest, and Naive Bayes. The selected model is chosen using phishing recall first and F1-score second because missing a phishing website is more dangerous than marking an extra site as suspicious.

Current model result. Best model: Random Forest | Model: Random Forest | Accuracy: 0.9463 | Phishing recall: 0.9446 | Phishing F1-score: 0.9446 | Confusion matrix: | [[3019 166] | [166 2833]] | Classification report: | precision recall f1-score support | | legitimate 0.95 0.95 0.95 3185 | phishing 0.94 0.94 0.94 2999

Evaluation Item	Result	Interpretation	Status
Selected model	Random Forest	Best model according to phishing recall and F1-score.	Passed
Accuracy	Approx. 94.5%	Correctly classifies most URLs in the held-out split.	Passed
Phishing recall	Approx. 94.9%	Prioritizes catching phishing URLs, reducing missed attacks.	Passed
Backend tests	8 passed	API, trusted domains, and feedback endpoint verified.	Passed
Extension checks	JS syntax passed	Popup, background, and content scripts parse correctly.	Passed

6. Backend API

The Flask backend exposes two main endpoints. The /predict endpoint receives a URL, checks trusted domains, extracts features, loads the saved model, and returns a prediction with risk score. The /mark-legitimate endpoint lets the user correct false positives.

- 1 Extension sends the current URL to /predict.
- 2 Backend checks trusted legitimate domains before calling the model.
- 3 If not trusted, features are extracted and passed to the trained model.
- 4 Backend returns prediction, risk score, URL, and source where applicable.
- 5 If the user marks a false positive as legitimate, the URL and domain are saved for future scans.

7. Browser Extension

The extension uses Chrome Manifest V3. Its background script monitors tab updates, calls the Flask backend, stores results, and notifies content scripts. The popup displays the scan result, risk score, and actions. The content script renders a full-page warning overlay for suspicious websites.

User Interface Behavior

- Safe sites show a green safe status and low risk score.
- Suspicious sites show a red danger status and warning text.
- The warning overlay includes Go Back, Mark as Legit, and Continue Anyway.
- Mark as Legit records user feedback and immediately trusts the corrected domain.
- Rescan Current Page sends the current URL to the backend again without retraining the model.

8. Testing

Testing covered feature extraction, backend endpoints, trusted-domain handling, user feedback, JavaScript syntax, and manual browser behavior.

- Feature tests verify URL normalization and suspicious signal extraction.
- Backend tests verify /health, /predict, trusted domains, and /mark-legitimate.
- JavaScript syntax checks verify background.js, popup.js, and content.js.
- Manual tests verify safe websites, suspicious phishing-like URLs, warning overlay actions, and user feedback.

9. Limitations

- The model uses URL-based features only and does not inspect full webpage HTML or screenshots.

- The extension depends on a locally running Flask backend.
- Trusted-domain overrides reduce false positives but must be maintained carefully to avoid over-trusting broad domains.
- The model is suitable for academic demonstration and should not be treated as production security software without further validation.

10. Future Improvements

- Add webpage content features such as forms, password fields, hidden inputs, and suspicious scripts.
- Use domain age, certificate information, WHOIS data, and reputation services.
- Add automated retraining after enough user feedback is collected.
- Package the model inside the extension using ONNX or TensorFlow.js to remove the Flask dependency.
- Add a dashboard showing scan history, blocked sites, and false-positive corrections.

11. Conclusion

The project demonstrates a complete AI-assisted phishing detection workflow. It combines a browser extension, Flask API, machine learning model, real phishing dataset, trusted-domain handling, user feedback, and clear warning actions. The result is an interactive cybersecurity tool that can identify risky URLs and help users avoid phishing attacks.

References

- ESDAUNG/PhishDataset public dataset: <https://github.com/ESDAUNG/PhishDataset>
- PhishTank phishing URL source referenced by the dataset: <https://phishtank.org>
- Scikit-learn machine learning library: <https://scikit-learn.org>
- Chrome Extensions Manifest V3 documentation: <https://developer.chrome.com/docs/extensions>
- Flask web framework documentation: <https://flask.palletsprojects.com>